



Inspiring Excellence

CSE340

Summer 2025

Assignment 1

Name : Abdullah Al Fahad

ID : 21201240

Section : 8

Faculty : PBK

Assignment 1

1. In computer architecture, "Performance via prediction" means improving performance by making guesses about what will happen next in program. CPU tries to predict the next step ^{based on history} and starts executing it ~~as it is~~ instead of waiting and this helps reduce delay and makes cpu faster.

Ex. `if (x > 10)`
`do something();`
`else`

`do something else();`

In here, value of x is unknown now based on history instead of waiting it predicts. Suppose $x > 10$ and start running `do something();`. If the prediction is correct.

time is saved. Moreover, if the prediction is wrong, cpu goes back and choose the connect path. now, ~~time is take~~ the time took for this backtime is called misprediction penalty.

2) Redundancy means keeping a backup system in case something fails. It is very important because it makes the system more reliable also it reduce the chance of total failure.

In some section like hospital, banking ^{section} ~~system~~ redundancy is very important because in these section downtime is not acceptable.

For example, Hospital stores patient data. If the main server fail and ~~then~~ if there is no backup, doctor's ~~to~~ can't access ~~to~~ data during emergencies.

such surgery. But if there is a redundant backup server, it can take over immediately. And the system runs smoothly and save lives.

③ a) Amdahl's law says if you make one part of task faster, it will improve the overall performance. The idea is, in some task there might some limitation like hardware limitation. If we improve the software part then the overall performance will improve.

Example: ¶ You designed a code and ~~code~~ your code takes 30 seconds to

Run. out of this 20 seconds are spent in a deeply nested loop and 10 seconds are spent elsewhere. After refactoring the

nested loop makes it 4 times faster

$20/4 \Rightarrow$ Only 5 seconds need. and total

time is taken. $(5+10) = 15$ seconds and this

makes the ~~small~~ overall performance 2 times

faster.

⑤ Amdahl's law relates closely to the design principal "Make the common case faster"

For example, AMD Ryzen 7 introduced

X3D processors which uses 3D V-cache to improve performance. In many games

the cpu frequently access memory. Because of L3 cache it stores commonly used

data and this significantly boost up the common case frequent memory access and

improve the average FPS even though the rest of the cpu architecture remains

same.

According to Amdahl's law this design improves overall performance because it

speeds up the part of the system which used most often.

4) Throughput is how many task a system can handle in a certain time and response time is how fast one task is completed. Increasing throughput can also improve response time when the system is overloaded.

For example, Star Cinplex ^{website} can handle only 50 ~~user~~ customers per second. Others are stuck in a loading screen. If they upgrade their system to handle 150 customers per second then the customers don't have to wait long so in this case, response

time for each user also improved.

5) When we measure a computer's speed ratio, it means how much the computer is faster compared to a reference computer. Moreover, a simple average can give a ~~wrong~~ misleading value if one value is too high and another is too small.

On the other hand, geometric mean gives a balanced overall performance. It avoids the distortion that arithmetic mean can introduce. Also, it reflects the real performance.

For example, PC 1 runs two test, test 1 got speed ratio 2 and in test 2 got speed ratio 0.5. In arithmetic mean it shows that the performance is improved which is not but in geometric mean it shows that 1 (no gain) which is the real scenario here.

6) Instruction count and CPI both are affected by the choice of programming language.

Different programming languages use different numbers of instructions when the code is converted to machine language. High-level language like Python, Java generators:

more instructions than the low level languages like C, assembly.

CPI depends on the type of instruction used, ~~some compi~~ different language instructions CPI will be different. ~~like,~~ like, python add instruction and assembly, add instruction ~~can~~ won't match the CPI. Therefore, CPI can be affected by the programming language.